JUNO

Final Year Project Report (CY4091 / CY4092)

Project Team

Syed Saad Mohsin 22I-1601
Muhammad Usman 22I-1689
Talat Faheem 22I-1735

Session 2022–2026

Supervised by

Mr. Arslan Aslam



Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

May, 2026

Contents

Abstract 1

Declaration 3

1 Introduction 5
- 1.1 Project Context and Actors . . . 5
- 1.2 Existing Solutions . . . 6
  - 1.2.1 Related Work and Academic Context . . . 7
- 1.3 Problem Statement . . . 7
- 1.4 Scope . . . 7
  - 1.4.1 In-Scope Features (Implemented) . . . 8
  - 1.4.2 Out-of-Scope or Current Constraints . . . 8
- 1.5 Modules . . . 9
  - 1.5.1 Identity, Access, and Workspace Module . . . 9
  - 1.5.2 Vendor-Supplier Collaboration Module . . . 9
  - 1.5.3 Commerce Sync and Operational Data Module . . . 9
  - 1.5.4 AI Helpdesk and Automation Module (JUNOCHAT / Juno Engine) 9
  - 1.5.5 Admin Observability and Governance Module . . . 10
- 1.6 Module Flow Diagrams . . . 11
  - 1.6.1 Overall State Flow . . . 11
  - 1.6.2 Core Sequence Flow . . . 12
  - 1.6.3 Level-0 Data Flow (DFD) . . . 13
- 1.7 Work Division . . . 13
- 1.8 Team Contributions . . . 14
  - 1.8.1 Syed Saad Mohsin (22I-1601) . . . 14
  - 1.8.2 Muhammad Usman (22I-1689) . . . 14
  - 1.8.3 Talat Faheem (22I-1735) . . . 15
  - 1.8.4 Applicant assessment . . . 15
- 1.9 User Class Table . . . 16

2 Project Requirements 17
- 2.1 Use-case / Event-Response / Storyboarding . . . 18
  - 2.1.1 Role-Centric Use-Case Interaction Diagram . . . 18

2.1.2 Core Use Cases . . . 19
2.2 Event-Response Table (Operationally Critical Events) . . . 21
2.3 Support Escalation Process Flow . . . 23
2.4 Detailed Event and State Behavior . . . 24
2.5 Functional Requirements . . . 25
2.5.1 Module A: Identity, Access, and Workspace . . . 25
2.5.2 Module B: Vendor-Supplier Collaboration . . . 26
2.5.3 Module C: Commerce Sync and Operational Data . . . 26
2.5.4 Module D: AI Helpdesk and Automation . . . 26
2.5.5 Module E: Admin Observability and Governance . . . 27
2.5.6 Functional Requirement Traceability . . . 27
2.6 Non-Functional Requirements . . . 27
2.6.1 Reliability . . . 27
2.6.2 Usability . . . 28
2.6.3 Performance . . . 28
2.6.4 Security . . . 28
2.7 Requirement Coverage Notes . . . 28

3 System Overview **29**
3.1 Architectural Design . . . 29
3.1.1 High-Level Box and Line View . . . 30
3.1.2 Layered Architecture Mapping . . . 30
3.1.3 Major Module Responsibilities . . . 31
3.1.4 Architecture Design Rationale . . . 31
3.2 Module Operational Design (DFD and State Flows) . . . 32
3.2.1 Identity, Access, and Workspace Module . . . 32
3.2.2 Commerce Sync and Data Flow (DFD) . . . 33
3.3 Process and Interaction Models . . . 33
3.3.1 Activity Diagram: Vendor Product Sync Lifecycle . . . 33
3.3.2 Sequence Diagram: AI Helpdesk and Escalation . . . 34
3.4 Data Design . . . 35
3.4.1 Data Storage Strategy . . . 35
3.4.2 Data Integrity and Consistency Design . . . 36
3.5 Domain Model . . . 36
3.5.1 Conceptual Domain ER Diagram . . . 36

4 Implementation and Testing **37**
4.1 Implementation Overview . . . 37
4.2 Algorithm Design . . . 37
4.2.1 Algorithm A: Product Sync Request Creation and Reopen Logic . . . 38
4.2.2 Algorithm B: Supplier Decision State Transition . . . 38

4.2.3 Algorithm C: Shopify Webhook Verification and Sync Metadata Update . . . 39
4.2.4 Algorithm Interaction Diagram . . . 40
4.3 External APIs and SDKs . . . 40
4.3.1 Local Model Training and Private Deployment Pipeline (future work) . . . 41
4.3.1.1 Candidate methodology: PEFT via LoRA . . . 41
4.4 Testing Details . . . 42
4.4.1 Testing Workflow . . . 42
4.4.2 Test Evidence Table . . . 42
4.4.3 QA Test Execution Summary . . . 43
4.4.4 Detailed Test Cases . . . 43

5 Conclusions and Future Work **49**
5.1 Conclusion . . . 49
5.2 Project Limitations . . . 50
5.3 Future Work . . . 50
5.3.1 Multi-Channel Communication Expansion . . . 50
5.3.2 Collaboration and Supply Intelligence . . . 50
5.3.3 Reliability and Scalability Enhancements . . . 51
5.3.4 On-Premises Model Adaptation . . . 51
5.4 Final Closing Statement . . . 51

References **54**

A Supplementary Material **55**
A.1 Work Division Reprint . . . 55
A.2 Glossary . . . 55
A.3 Selected Test Identifier Index . . . 56
A.4 Exported Architecture Views . . . 56

List of Figures

1.1 Overall State Flow Diagram 11
1.2 Core Sequence Flow Diagram 12
1.3 Level-0 Data Flow Diagram (DFD) 13
1.4 Team ownership overlay on the implemented JUNO layers (same split as Table 1.3). 13

2.1 Role-Centric Use-Case Interaction Diagram 18
2.2 Role-centric context view of the implemented JUNO platform (vendor, supplier, admin, and end-customer message flows). 18
2.3 Support Escalation Process Flow 23
2.4 Product Sync State Transitions 24
2.5 Product Sync State Transitions 25

3.1 High-Level Box and Line View 30
3.2 Layered Architecture Mapping 30
3.3 Implemented module map (identity, collaboration, commerce sync, AI helpdesk, governance). Arrows indicate typical operational coupling, not a strict pipeline. 31
3.4 Identity and Access State Flow 32
3.5 Data Flow Diagram: Commerce Sync Module (HMAC-verified Shopify events into the internal cache). 33
3.6 Vendor Product Sync Lifecycle Activity Diagram 34
3.7 AI helpdesk sequence: storefront message, RAG context lookup, draft reply, and optional escalation ticket. 35
3.8 Conceptual Domain ER Diagram 36

4.1 Interaction between System Events and Implementation Algorithms . . . 40
4.2 JUNO Verification and Testing Workflow 42

A.1 Exported data-flow view of JUNO modules (source export). 57
A.2 Exported data-flow view (alternate layout; source export). 58

List of Tables

1.1 Core Roles in JUNO 6
1.2 Comparison of Existing Solutions 6
1.3 Work Division 13
1.4 Final Year Project Assessment (Syed Saad Mohsin) 15
1.5 User Classes 16

2.1 Event-Response Table 22
2.2 Traceability Matrix 27

3.1 Major Module Responsibilities 31
3.2 Data Storage Strategy 35

4.1 External APIs and SDKs 41
4.2 Test Evidence Table 42
4.3 QA Test Execution Summary 43
4.4 Test Case: Role-Based Access Control (RBAC) Integrity 43
4.5 Test Case: Product Sync Request Reopen Logic (Algorithm A) 44
4.6 Test Case: Supplier Decision State Transition (Algorithm B) 44
4.7 Test Case: Shopify Webhook HMAC Verification (Algorithm C) 44
4.8 Test Case: AI RAG-based Support Response 45
4.9 Test Case: Automated Ticket Escalation 45
4.10 Test Case: Gmail Webhook Auto-Reply Pipeline 45
4.11 Test Case: Multi-Factor Authentication (MFA) Verification 46
4.12 Test Case: Shopify Store Onboarding and App Proxy 46
4.13 Test Case: Sync Metadata Tracking and Failure Logging 46
4.14 Test Case: Vendor-Specific Product Offer Creation 47
4.15 Test Case: Anti-Abuse Rate Limiting 47
4.16 Test Case: Admin Event Stream Visibility 47

A.1 Work Division (reprint) 55
A.2 Selected test identifiers 56

Abstract

This report documents *JUNO*, a role-based commerce operations platform developed as the Final Year Project (CY4091/CY4092) for the B.Sc. Cyber Security programme at FAST-NUCES, Islamabad. The system integrates vendor, supplier, and admin workflows for Shopify-centered dropshipping operations through a unified, event-driven architecture.

Operational fragmentation—scattered communication, stale product data, and weak auditability—is addressed by combining structured vendor–supplier collaboration, webhook-verified commerce synchronisation, AI-assisted customer support (JUNOCHAT), and admin observability. The platform enforces role-based access control (RBAC) across three operational surfaces, links sync requests and product offers to a shared data model, and applies security-oriented non-functional requirements including MFA, HMAC webhook verification, and escalation paths for sensitive cases.

The implementation follows a layered architecture with distinct modules for identity and workspace management, vendor–supplier collaboration, commerce sync, AI helpdesk automation, and governance/observability. Testing covers RBAC integrity, sync state transitions, webhook verification, AI helpdesk end-to-end and safety cases, and cross-role user journeys.

JUNO was delivered by a three-person team (see Table 1.3 and Section 1.8): Syed Saad Mohsin (role-gated interfaces, JUNOCHAT integration into operator workflows, and verification), Muhammad Usman (AI orchestration, retrieval/provider behaviour, and data model), and Talat Faheem (APIs, webhook handlers, and deployment).

Keywords: role-based access control, event-driven architecture, workflow systems, webhook security, AI helpdesk, software engineering, systems testing.

Declaration

We declare that this Final Year Project report titled *JUNO* is submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Cyber Security at the National University of Computer and Emerging Sciences (FAST-NUCES), Islamabad Campus.

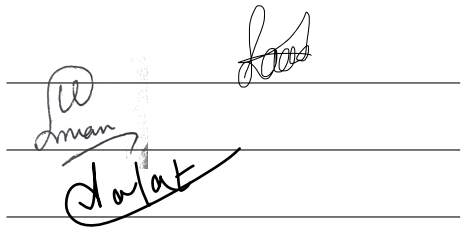
This is a **group project**. All team members contributed to the overall system design and documentation. Individual contributions are stated in Section 1.8 and Table 1.3.

We confirm that this report represents our own work except where references to published sources are made. We understand that plagiarism or misrepresentation of contributions may result in disciplinary action.

Syed Saad Mohsin (22I-1601)

Muhammad Usman (22I-1689)

Talat Faheem (22I-1735)



Supervisor: Mr. Arslan Aslam **Date:** May 2026

Chapter 1

Introduction

This report presents *JUNO*, a Final Year Project (CY4091/CY4092) submitted in partial fulfilment of the B.Sc. Cyber Security degree at FAST-NUCES, Islamabad (Session 2022–2026). JUNO is a role-based commerce operations platform for Shopify-centered dropshipping businesses. The implemented system combines three operational surfaces—a vendor portal, a supplier portal, and an admin observability area—with AI-assisted communication and ticketing workflows connected to store data.

In many small and mid-size dropshipping operations, communication is scattered across inboxes, chat threads, and spreadsheets. This creates delays in supplier coordination, missing context in customer responses, stale product/price information, and poor traceability of decisions. JUNO addresses this by enforcing structured relationships between vendors and suppliers, maintaining synchronised commerce data, and adding AI features where repetitive communication causes the highest operational load.

Unlike a standalone chatbot or a basic ticket dashboard, JUNO is implemented as a workflow system: users, stores, supplier profiles, connections, products, sync requests, offers, notifications, and AI-assisted replies are all linked through a single data model. This allows the platform to support both business operations (inventory/offers/sync status) and support operations (storefront chat, Gmail handling, ticket escalation) in one environment.

1.1 Project Context and Actors

The system defines and enforces three core roles as outlined in Table 1.1.

Table 1.1: Core Roles in JUNO

Actor	Description	Main Responsibilities in JUNO
Vendor	Shopify merchant/store owner using JUNO as the operational center.	Onboard store credentials, monitor products and orders, connect with suppliers, request product sync, use AI support tools, and handle customer-side workflows.
Supplier	Product provider connected to one or more vendors.	Manage supplier catalog, accept/reject product sync requests, create/update vendor-specific offers, and respond to vendor coordination needs.
Admin	Platform operations and governance role.	Observe system health, events, connections, product sync trends, order-level signals, and email activity (observability-focused behavior).

1.2 Existing Solutions

A review of currently used approaches shows that existing tools usually solve one part of the workflow but not the complete vendor-supplier-support loop. Table 1.2 provides a comparison of JUNO against traditional methods.

Table 1.2: Comparison of Existing Solutions

System Name	System Overview	System Limitations Compared to JUNO
Generic Helpdesk Platforms	Centralize incoming support conversations and ticket assignments.	Limited direct linkage to vendor-supplier product workflows, weak per-store commerce context, and no native supplier collaboration model.
Spreadsheet + Messaging Operations	Teams coordinate pricing, stock, and issue resolution in sheets plus chat apps.	No reliable source of truth, high risk of outdated values, poor auditability, and heavy manual follow-up.
Single-Function Shopify Apps	Apps offer one feature such as product sync, notifications, or chatbot widgets.	Fragmented architecture; weak cross-feature continuity between product offers, ticket escalation, and AI-driven communication.

1.2.1 Related Work and Academic Context

Prior work on supply-chain coordination on the Internet emphasises the need for integrated marketing–operations systems rather than isolated tools [6, 11]. Role-based access control (RBAC) provides a well-established model for enforcing least-privilege separation in multi-actor systems [2, 15]. Event-driven and webhook-based architectures, as used in modern commerce platforms, require integrity verification and reliable state synchronisation [3, 7]. JUNO applies these principles to a concrete dropshipping workflow, combining RBAC-gated interfaces, webhook-verified Shopify integration, and structured testing of cross-role behaviour.

1.3 Problem Statement

Shopify-based dropshipping operations depend on fast and accurate communication between customers, vendors, and suppliers, but these interactions are often spread across disconnected tools. Customer conversations may happen in email or storefront chat, vendor actions in internal dashboards, and supplier coordination in informal message threads. This separation causes context loss, delayed updates, and inconsistent decision-making.

The project identifies five core problems:

1. **Communication fragmentation** increases response time and handoff errors.
2. **Repetitive support workload** (order status, shipping, returns, cancellations) consumes team capacity and reduces consistency under high volume.
3. **Weak vendor-supplier alignment** in product synchronization and offer updates leads to outdated price/quantity data and margin risk.
4. **Limited prioritization and risk handling** causes critical conversations to be treated like routine ones.
5. **Insufficient role-aware visibility** makes it harder to monitor operational health and improve processes proactively.

Together, these issues reduce operational efficiency and customer satisfaction. JUNO addresses them through a unified, role-based platform that combines structured collaboration, commerce data workflows, AI-assisted communication, and monitoring, while keeping human oversight for sensitive cases.

1.4 Scope

The scope of this project defines what will be included and excluded, helping to set clear expectations for the reader. This section outlines the boundaries and limitations of the

development project.

1.4.1 In-Scope Features (Implemented)

- Multi-role authentication and role-gated workspaces for vendor, supplier, and admin usage.
- Vendor store onboarding with Shopify credentials and webhook registration.
- Supplier connection lifecycle and invitation/approval style relationship management.
- Product collaboration workflows:
  - Vendor product sync requests.
  - Supplier-side sync decisions.
  - Supplier product offers with vendor-specific values.
- Commerce cache handling for products, orders, and customers from Shopify webhooks.
- Notification system for operational events (including sync and offer changes).
- AI-powered support layer including:
  - Storefront chat message handling.
  - Knowledge base retrieval with vector search.
  - Gmail webhook-based auto-reply pipeline.
  - Ticket generation/escalation paths.
- Admin observability dashboards for system-level monitoring.

1.4.2 Out-of-Scope or Current Constraints

- Full social-channel unification (all messaging channels are not fully production-integrated).
- Complete enterprise BI/reporting suite beyond current operational dashboards.
- High-throughput asynchronous ingestion for all knowledge workflows (some processing remains request-time).

1.5 Modules

The implemented architecture can be described through five major modules. The modules are logically organized to reflect the overall system workflow.

1.5.1 Identity, Access, and Workspace Module

This module handles user identity, role separation, and active-store context required by other modules.

- Role-based access for vendor, supplier, and admin flows.
- Store-scoped operations for vendor actions.
- Foundation for role-aware API guards and layout routing.

1.5.2 Vendor-Supplier Collaboration Module

This is the operational core that formalizes business relationships and product-level collaboration.

- Vendor-supplier connection and invitation lifecycle.
- Product sync request workflow (`PENDING`, `ACCEPTED`, `REJECTED`).
- Supplier product offers with per-vendor pricing/quantity control.
- Change visibility for vendors through notifications and new-product style views.

1.5.3 Commerce Sync and Operational Data Module

This module keeps JUNO aligned with Shopify events and provides normalized internal data for workflows and AI features.

- Shopify webhook verification and event consumption.
- Cached product/order/customer updates for store context.
- Sync metadata tracking and health-oriented operational signals.
- Trigger points for inventory and notification logic.

1.5.4 AI Helpdesk and Automation Module (JUNOCHAT / Juno Engine)

This module provides AI-assisted communication across storefront and email channels using configurable providers and retrieval workflows.

- Storefront conversation handling with AI response generation.

- Knowledge base indexing and query-time semantic retrieval using embeddings.
- Gmail inbound processing with classification, safety checks, and auto-reply behavior.
- Ticket and escalation workflow integration for unresolved or sensitive cases.
- Supplier-contact automation from ticket/order context in selected flows.

1.5.5 Admin Observability and Governance Module

This module offers cross-system visibility for operations monitoring and diagnosis.

- Overview of stores, connections, sync activity, orders, and email events.
- Event and trend visibility for platform operations.
- Read-oriented governance layer for monitoring and auditing.

1.6 Module Flow Diagrams

This section presents concise visual models for the implemented modules.

1.6.1 Overall State Flow

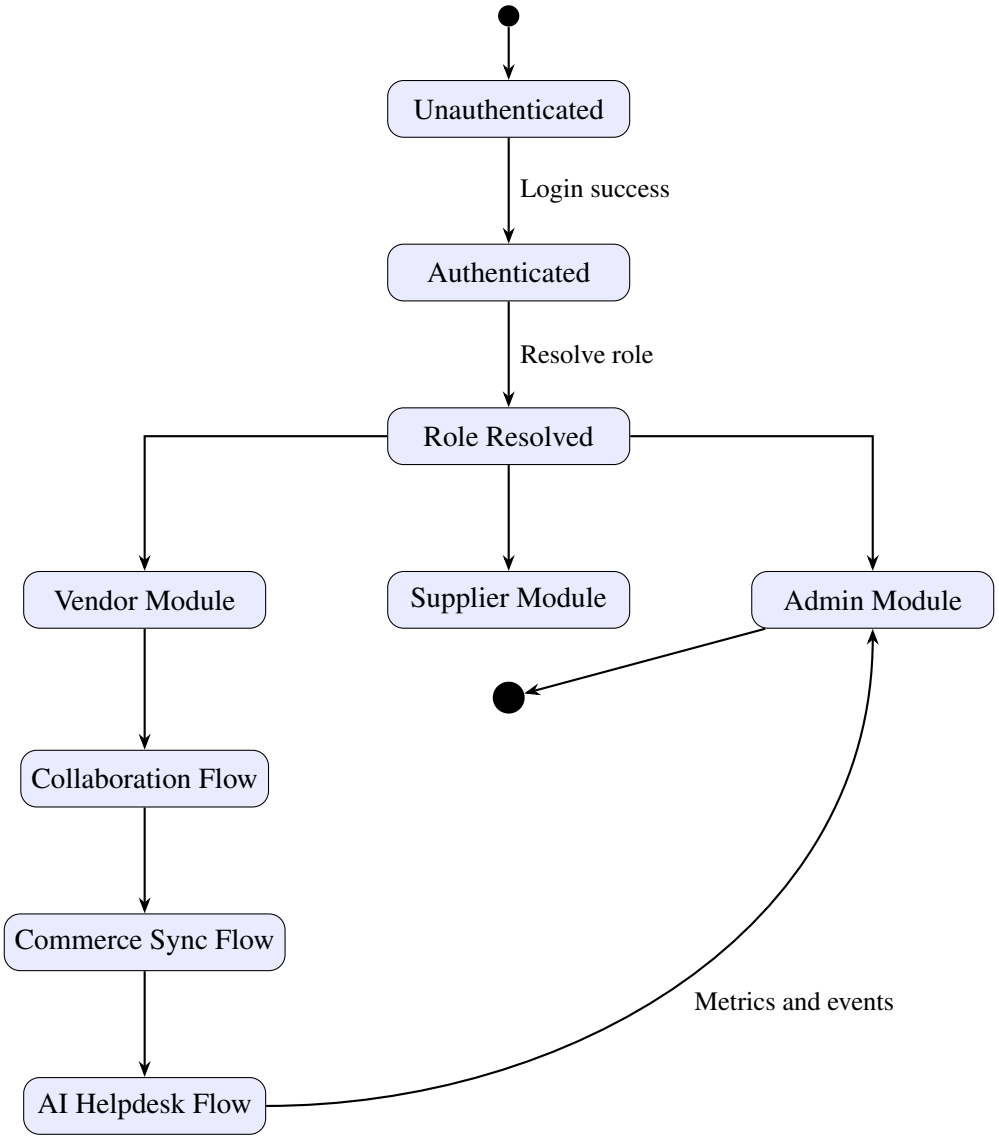


Figure 1.1: Overall State Flow Diagram

1.6.2 Core Sequence Flow

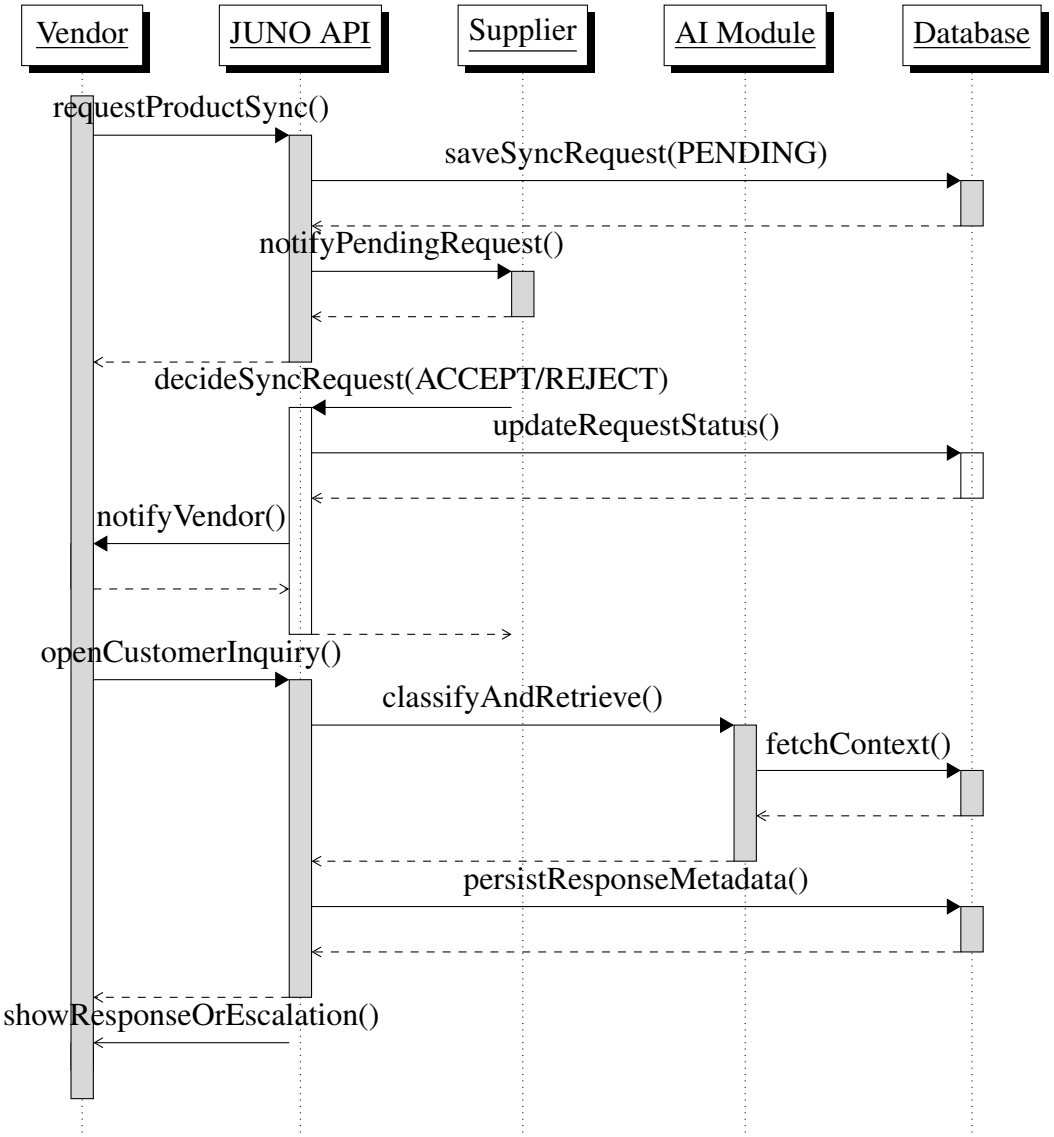


Figure 1.2: Core Sequence Flow Diagram

1.6.3 Level-0 Data Flow (DFD)

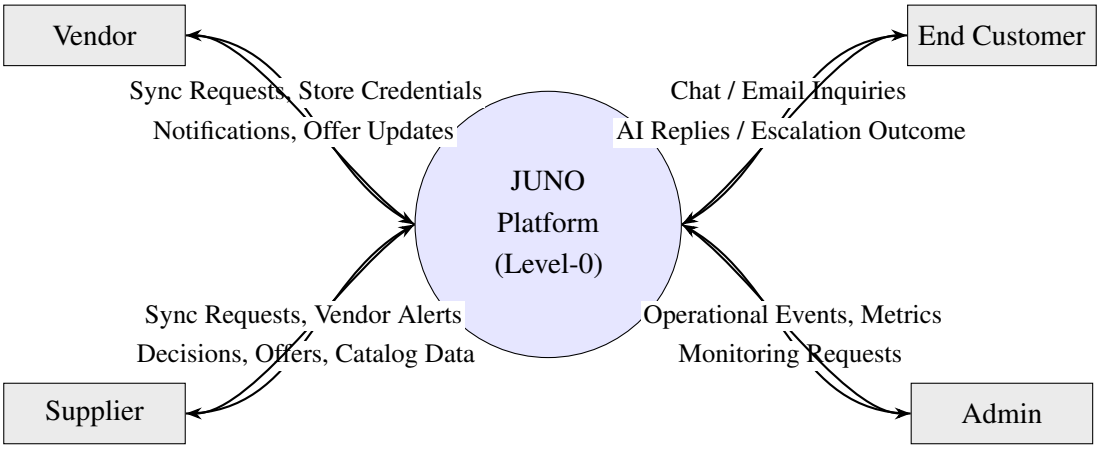


Figure 1.3: Level-0 Data Flow Diagram (DFD)

1.7 Work Division

The following work-division model reflects responsibilities typically needed to deliver the currently implemented system.

Table 1.3: Work Division

Team Member	**Responsibilities**
Syed Saad Mohsin	Build vendor/supplier/admin interfaces and role-based UX; integrate AI helpdesk (JUNOCHAT) into operator workflows; execute UI/functional, security, and AI end-to-end testing across major user journeys.
Talat Faheem	Implement APIs, business workflows, webhook handlers, and manage deployment, environment, and CI/CD operations.
Muhammad Usman	Lead AI orchestration/retrieval/provider behaviour and notifications; manage Prisma schema, relations, migrations, and data consistency.

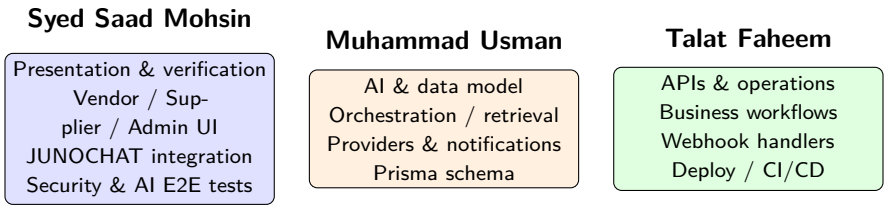


Figure 1.4: Team ownership overlay on the implemented JUNO layers (same split as Table 1.3).

1.8 Team Contributions

This section expands Table 1.3 for academic assessment. JUNO is a group Final Year Project; the three scopes below are complementary and describe the *implemented* system only.

1.8.1 Syed Saad Mohsin (22I-1601)

Responsibilities covered the **role-gated application surfaces**, **AI helpdesk integration into those surfaces**, and the **verification layer**:

- Design and implementation of vendor, supplier, and admin web interfaces, including role-gated navigation and store-scoped workspace layouts.
- Role-based UX flows ensuring that each actor accesses only permitted operations, consistent with RBAC models [2, 15], including surfaces that expose AI support tools to the correct roles.
- Vendor–supplier product-sync and notification UI surfaces integrated with backend workflows.
- Substantial contribution to the **AI-assisted support layer (JUNOCHAT)**: storefront chat and support UI, wiring AI responses into operator workflows, escalation and ticket paths for human takeover, and end-to-end checks that AI features respect role boundaries and store scope.
- Functional, integration, security, and AI-related testing across major user journeys (Chapter 4, Section 4.4).

Test cases in Section 4.4 primarily authored or executed by this member include: RBAC Integrity; MFA Verification; Shopify Webhook HMAC Verification; Anti-Abuse Rate Limiting; Admin Event Stream Visibility; cross-role functional tests (vendor onboarding, sync-request UI, supplier decision flows); AI RAG-based support response (end-to-end retrieved-context answers); AI escalation and safety cases; Gmail auto-reply / helpdesk automation UAT.

This scope spans software systems (multi-actor RBAC platforms), integration (commerce-sync UI and cross-module journeys), and applied AI at the application/verification layer (helpdesk automation with retrieval-backed responses, escalation, and safety checks) [7, 12].

1.8.2 Muhammad Usman (22I-1689)

Responsibilities covered the **AI orchestration and data-model** layer:

- AI orchestration for storefront and email support, including retrieval/provider behaviour and prompt-side configuration for JUNOCHAT.
- Notification behaviour tied to collaboration and support events.
- Prisma schema, relations, migrations, and data consistency for stores, connections, sync records, offers, tickets, and knowledge chunks (including vector retrieval columns used by RAG).

This member leads the backend AI/retrieval path that the helpdesk UI and end-to-end tests consume. Local LoRA training is *not* part of the shipped system (Chapter 4 records it as future work).

1.8.3 Talat Faheem (22I-1735)

Responsibilities covered the **API, integration, and operations** layer:

- Application APIs and business workflows for collaboration and commerce synchronisation.
- Webhook handlers for external commerce events (used together with HMAC verification on the ingestion path).
- Deployment, environment configuration, and CI/CD operations for the shared Next.js application.

This member leads the service and operations path that role-gated UIs and AI pipelines call.

1.8.4 Applicant assessment

Table 1.4: Final Year Project Assessment (Syed Saad Mohsin)

Course	**Title**	**Semester**	**Grade**
CY4091	Final Year Project I	Fall 2025	B
CY4092	Final Year Project II	Spring 2026	A−

1.9 User Class Table

Table 1.5 summarizes the primary user classes that interact with JUNO and how they use the platform.

Table 1.5: User Classes

User Class	**Description**	**Key Interactions with System**
Vendor	Shopify store owner or operator using JUNO for day-to-day dropshipping operations.	Onboarding store, viewing products/orders/customers, requesting product sync, coordinating with suppliers, handling AI-assisted support workflows.
Supplier	Product fulfillment partner connected to one or more vendors.	Managing supplier catalog, responding to sync requests, sending/updating product offers, and coordinating order-related communication with vendors.
Admin	Internal operations/governance user with platform-wide visibility.	Monitoring stores, events, sync activity, and email/system health through observability-focused admin dashboards.
End Customer	External customer purchasing from a vendor's Shopify store.	Interacts through storefront chat or email channels that are processed by JUNO's support automation and ticketing workflows.

Chapter 2

Project Requirements

This section defines the project requirements for JUNO. It translates the project context from Section 1 into concrete user interactions, functional requirements, and measurable non-functional requirements for the implemented scope.

The objective of this chapter is to establish a clear baseline for how the system is expected to behave from both user and operational perspectives. It identifies the major actors, core workflows, and event-response behavior that drive the platform in real usage scenarios.

2.1 Use-case / Event-Response / Storyboarding

JUNO supports multi-role interactions between Vendor, Supplier, Admin, and End Customer (external actor through storefront chat and email channels). The following use cases capture the most critical operational and support flows.

2.1.1 Role-Centric Use-Case Interaction Diagram

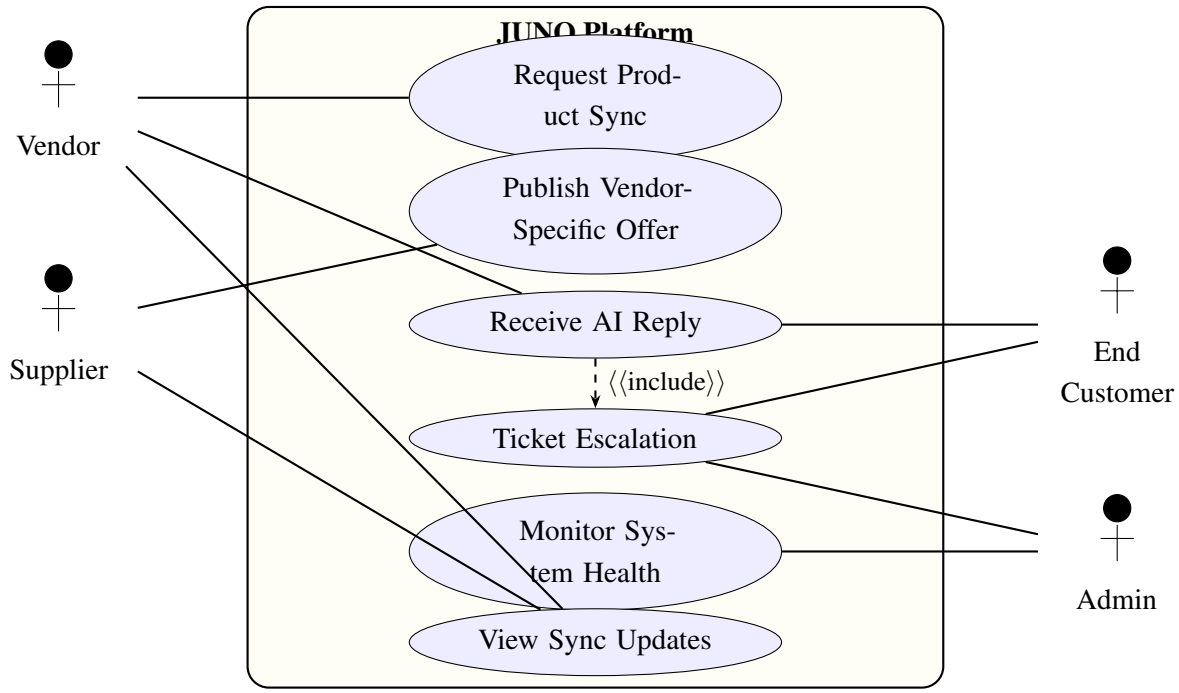


Figure 2.1: Role-Centric Use-Case Interaction Diagram

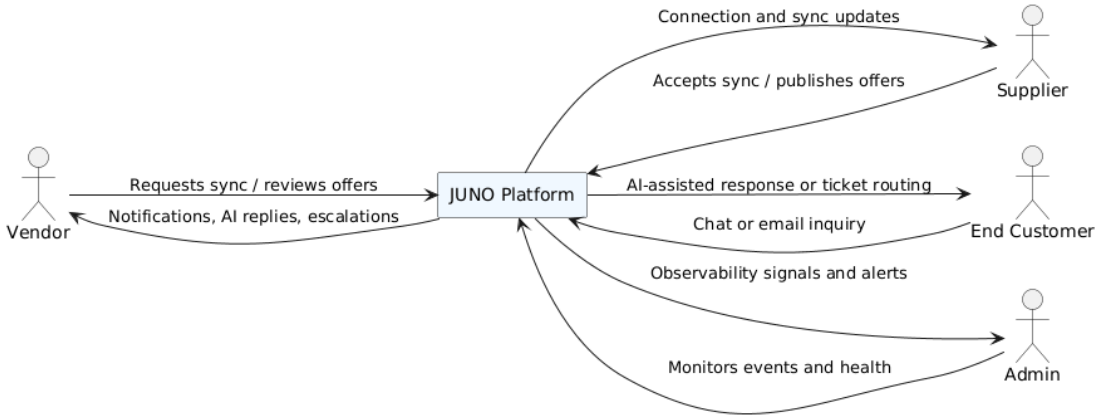


Figure 2.2: Role-centric context view of the implemented JUNO platform (vendor, supplier, admin, and end-customer message flows).

2.1.2 Core Use Cases

UC-01: Vendor Requests Product Sync from Supplier

- **Primary Actor:** Vendor
- **Supporting Actor:** Supplier
- **Preconditions:** Vendor account and store are onboarded; vendor-supplier connection exists; target supplier product is discoverable.
- **Trigger:** Vendor selects supplier product and initiates sync request.
- **Main Flow:**
  1. Vendor opens supplier product details.
  2. Vendor submits sync request to supplier.
  3. System stores request with pending status.
  4. Supplier receives notification and reviews request.
  5. Supplier accepts request.
  6. System updates status and exposes synced product for vendor operations.
- **Alternate/Error Flows:**
  - Supplier rejects request; system records rejection reason and notifies vendor.
  - Request times out without action; system marks stale status and keeps vendor informed.
- **Postconditions:** Request status is finalized as accepted or rejected; both roles can audit the decision trail.

UC-02: Supplier Publishes Vendor-Specific Offer

- **Primary Actor:** Supplier
- **Supporting Actor:** Vendor
- **Preconditions:** Active vendor-supplier relationship exists; relevant product has synced or is eligible for offer mapping.
- **Trigger:** Supplier creates or updates an offer for a specific vendor.
- **Main Flow:**

1. Supplier selects vendor and product.
2. Supplier enters vendor-specific price, quantity, and terms.
3. System validates offer constraints and saves versioned offer.
4. Vendor receives notification and views updated offer details.

- **Alternate/Error Flows:**
  - Invalid pricing/quantity values are rejected with corrective guidance.
  - Offer conflicts with existing state; system requests explicit overwrite confirmation.
- **Postconditions:** Vendor-visible offer is updated and available for decision-making.

UC-03: Customer Inquiry is Auto-Handled with Escalation Safety

- **Primary Actor:** End Customer (external)
- **Supporting Actors:** Vendor support operator, AI helpdesk pipeline
- **Preconditions:** Chat/email channel is connected; retrieval content and policy checks are available.
- **Trigger:** Customer sends a message about order status, return, shipping, or cancellation.
- **Main Flow:**
  1. System receives inbound message.
  2. Message is classified and normalized.
  3. Knowledge retrieval and contextual checks run.
  4. AI drafts or sends response for low-risk scenarios.
  5. If confidence/risk thresholds fail, system creates or updates escalation ticket.
  6. Vendor support operator handles escalated case.
- **Alternate/Error Flows:**
  - Missing context leads to a clarification question instead of direct resolution.
  - Safety or policy violation triggers immediate human-only handling.
- **Postconditions:** Customer receives a timely response, with risky cases routed to human review.

UC-04: Admin Monitors Operational Health

- **Primary Actor:** Admin
- **Preconditions:** Admin access is active; observability data is available.
- **Trigger:** Admin opens monitoring dashboard.
- **Main Flow:**
  1. Admin reviews stores, sync trends, and communication events.
  2. Admin inspects anomalies (e.g., failed sync spikes, unresolved escalations).
  3. Admin drills into event streams for diagnosis and governance.
- **Postconditions:** Admin captures actionable insights and flags issues for operational follow-up.

2.2 Event-Response Table (Operationally Critical Events)

The Event-Response Table maps high-impact operational triggers to expected system behavior. It provides a compact view of how JUNO reacts to real-time events generated by vendors, suppliers, customers, automation pipelines, and admin users. This mapping helps validate that each critical trigger has a defined response path, clear ownership, and a measurable business outcome.

Table 2.1: Event-Response Table

ID	Event / Stimulus	Source	System Response	Outcome
EVT-01	Vendor submits sync request	Vendor	Create pending record, notify supplier, log event	Supplier receives request
EVT-02	Supplier accepts sync	Supplier	Mark accepted, update availability, notify vendor	Product collaboration continues
EVT-03	Supplier rejects sync	Supplier	Mark rejected, attach reason, notify vendor	Vendor gets rejection context
EVT-04	New offer created	Supplier	Validate and persist, emit notification	Vendor sees latest terms
EVT-05	Customer sends message	Customer	Classify, run retrieval, AI reply or escalate	Fast first response
EVT-06	Confidence low	AI Pipeline	Create escalation ticket, attach context	Human support takes over
EVT-07	Webhook sync failure	Subsystem	Record failure, retry, notify operational channel	Reliability protections trigger
EVT-08	Admin reviews anomaly	Admin	Provide filtered event trail and entities	Faster diagnosis

2.3 Support Escalation Process Flow

The support escalation process flow describes how JUNO balances automation speed with operational safety. Each inbound support message first passes through intent/risk classification and contextual retrieval so that the system can decide whether a direct AI response is appropriate or whether human intervention is required.

This flow is designed to reduce response latency for routine cases while protecting service quality in uncertain or sensitive scenarios. The decision point (confidence and safety pass) acts as a control gate to ensure that unresolved, risky, or low-confidence cases are routed to controlled escalation rather than auto-resolved incorrectly.

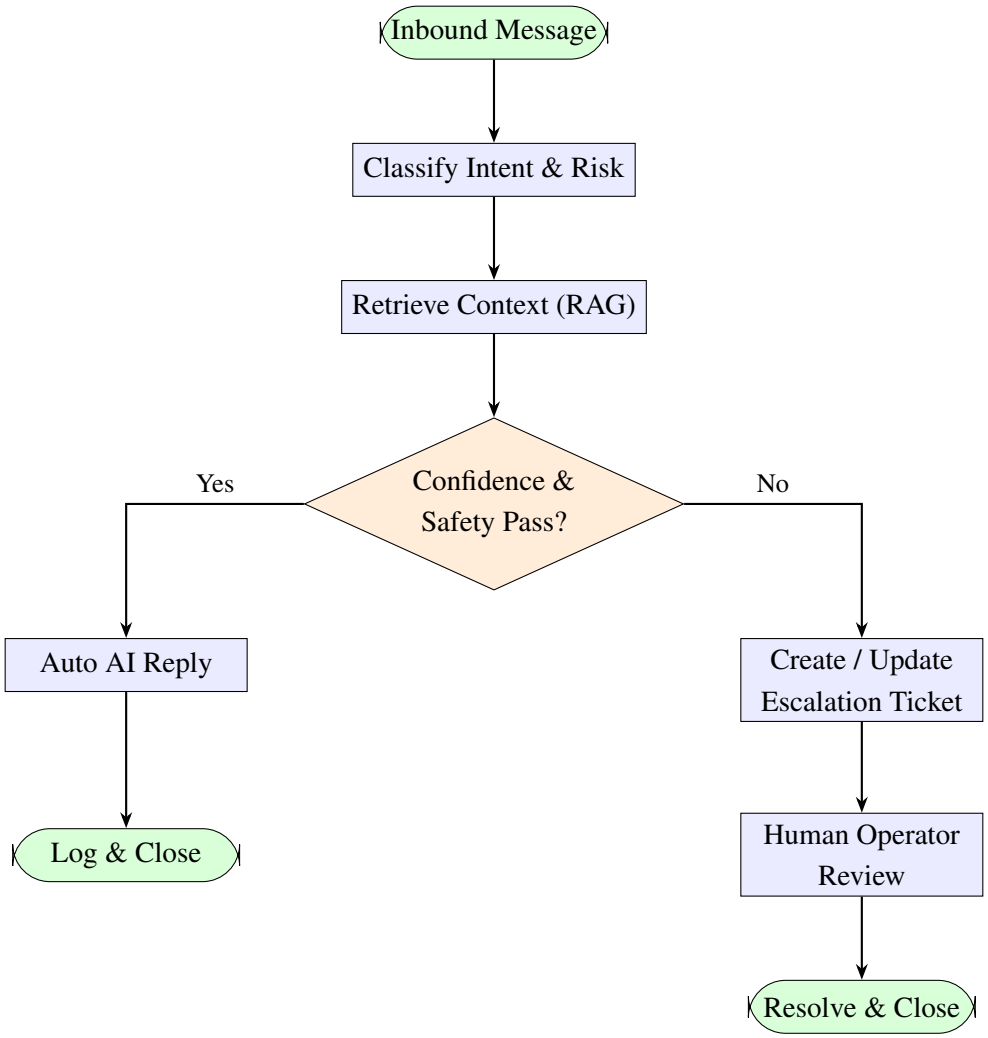


Figure 2.3: Support Escalation Process Flow

2.4 Detailed Event and State Behavior

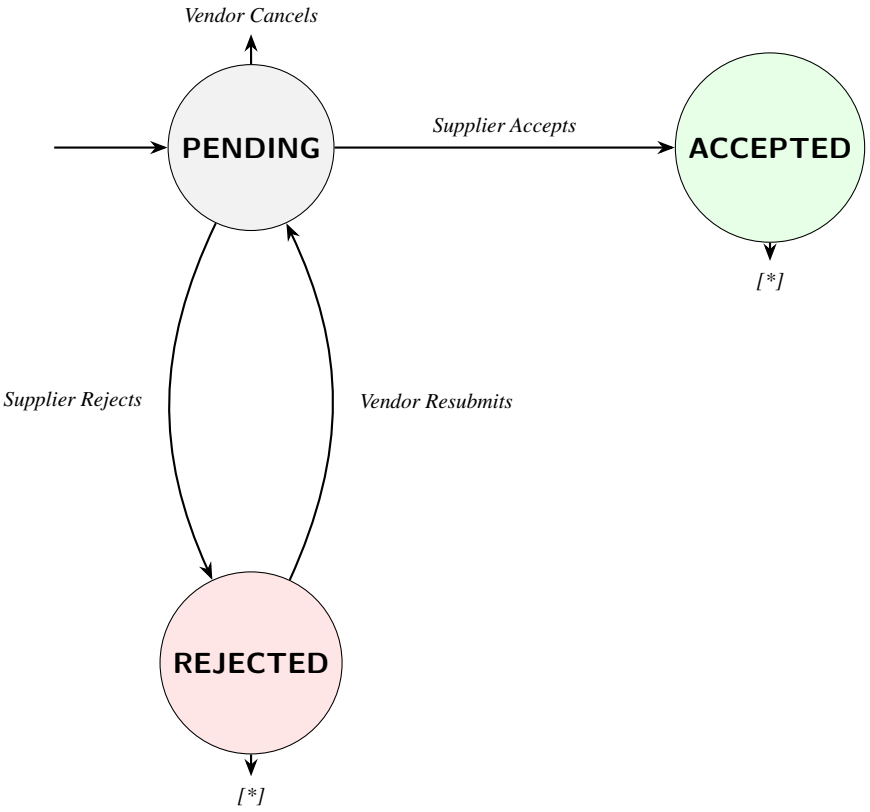


Figure 2.4: Product Sync State Transitions

Product Sync State Transitions Behavior notes:

- Sync request uniqueness is scoped to `storeId + supplierId + shopifyProductId`.
- A previously rejected sync can be reopened as `PENDING` instead of creating a duplicate row.
- Suppliers can decide pending requests, but cannot delete a pending request (they must reject).
- Accepted sync records preserve supplier-available quantity snapshot in product data.

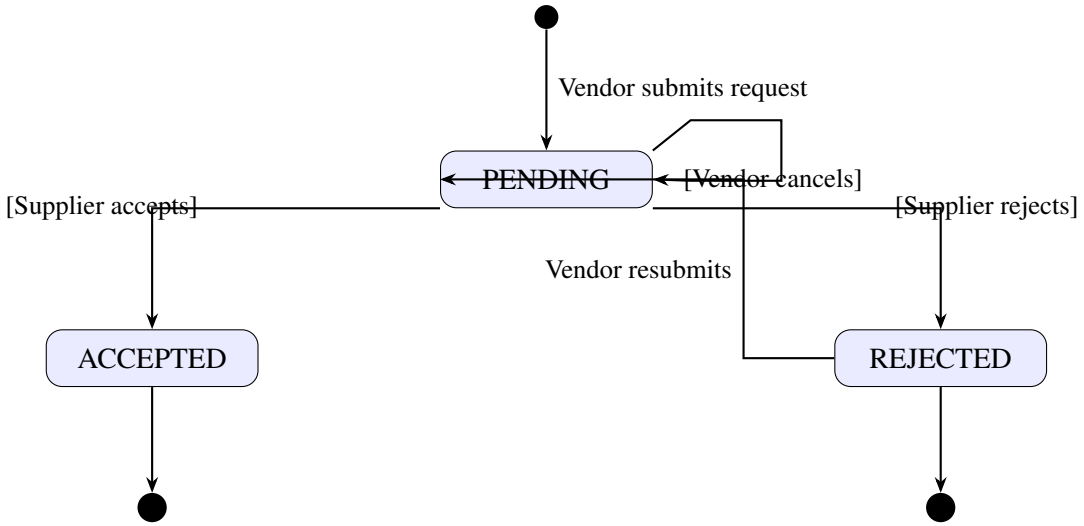


Figure 2.5: Product Sync State Transitions

Storefront Support Escalation Trigger Behavior

- Escalation ticket creation is triggered when AI output indicates explicit escalation intent.
- Escalation artifacts are persisted through `EmailLog` with trigger classification.
- Existing escalation thread keys are reused to avoid duplicate ticket spam.

2.5 Functional Requirements

Functional requirements are organized by implemented modules.

2.5.1 Module A: Identity, Access, and Workspace

ID	Functional Requirement
FR-IAM-01	Authenticate users and bind account to role (`Vendor`, `Supplier`, `Admin`).
FR-IAM-02	Enforce RBAC on routes, APIs, and workspace features.
FR-IAM-03	Maintain active store context for vendor operations.
FR-IAM-04	Prevent unauthorized cross-role data visibility.
FR-IAM-05	Record auth decision events for auditability.

2.5.2 Module B: Vendor-Supplier Collaboration

ID	Functional Requirement
FR-COLLAB-01	Allow vendors to create supplier invitations and track status.
FR-COLLAB-02	Allow vendors to submit product sync requests.
FR-COLLAB-03	Allow suppliers to accept or reject sync requests.
FR-COLLAB-04	Allow suppliers to create vendor-specific offers.
FR-COLLAB-05	Notify affected vendors on status or offer changes.
FR-COLLAB-06	Preserve history of collaboration events.

2.5.3 Module C: Commerce Sync and Operational Data

ID	Functional Requirement
FR-SYNC-01	Verify and process inbound commerce webhooks.
FR-SYNC-02	Update cached records for products, orders, and customers.
FR-SYNC-03	Maintain sync metadata for diagnostics.
FR-SYNC-04	Emit notifications for significant sync outcomes.
FR-SYNC-05	Expose synced data to collaboration and AI modules.

2.5.4 Module D: AI Helpdesk and Automation

ID	Functional Requirement
FR-AI-01	Ingest chat messages and inbound emails into a unified support pipeline.
FR-AI-02	Classify each incoming message by support intent (e.g., order status, returns, shipping, cancellation).
FR-AI-03	Perform Retrieval-Augmented Generation (RAG) context lookup against the knowledge base before response generation.
FR-AI-04	Apply confidence scoring and safety policy checks before dispatching any automated reply.
FR-AI-05	Automatically escalate high-risk or low-confidence cases to a human-reviewable ticket.
FR-AI-06	Allow support operators to review, respond to, and resolve escalated tickets.
FR-AI-07	Log all AI-generated outcomes, classifications, and escalation decisions for monitoring and tuning.

2.5.5 Module E: Admin Observability and Governance

ID	Functional Requirement
FR-OBS-01	Provide consolidated view of stores, syncs, and events.
FR-OBS-02	Expose trend indicators for operational anomalies.
FR-OBS-03	Support drill-down into entity timelines.
FR-OBS-04	Restrict admin actions to governance behavior.
FR-OBS-05	Retain observability records for audit period.

2.5.6 Functional Requirement Traceability

Table 2.2: Traceability Matrix

FR ID	Module	API / Surface	Key Entities	Test Case ID
FR-IAM-01–05	Identity & Access	Auth routes / guards	User, Session, Account	TC-IAM-01, TC-IAM-02
FR-COLLAB-01–06	Vendor-Supplier	/api/products/sync	Connection, ProductSync, Offer	TC-COLLAB-01–03
FR-SYNC-01–05	Commerce Sync	/api/webhooks/shopify	CachedProduct, SyncMetadata	TC-SYNC-01–02
FR-AI-01–07	AI Helpdesk	/api/storefront-chat	EmailLog, FlaggedEmail	TC-AI-01–04
FR-OBS-01–05	Admin Observability	/api/admin/events	SyncMetadata, Connection	TC-ADMIN-01

2.6 Non-Functional Requirements

2.6.1 Reliability

ID	Requirement
NFR-REL-01	Target monthly service availability of at least 99.0%.
NFR-REL-02	Retry critical webhook processing with bounded logic.
NFR-REL-03	Preserve ticket history for post-incident traceability.
NFR-REL-04	Fail safely by escalating to human handling on degradation.
NFR-REL-05	Webhook endpoints shall favor idempotency.

2.6.2 Usability

ID	Requirement
NFR-USA-01	Users complete first core task with minimal friction.
NFR-USA-02	Keep role-specific navigation consistent across sessions.
NFR-USA-03	Provide immediate visual feedback for workflow actions.
NFR-USA-04	Error states shall provide actionable recovery paths.
NFR-USA-05	Role-scoped views expose only task-relevant actions.

2.6.3 Performance

ID	Requirement
NFR-PERF-01	95% of interactive requests complete in $\leq$ 3 seconds.
NFR-PERF-02	AI classification shall sustain near real-time behavior.
NFR-PERF-03	Notifications shall be near-immediate.
NFR-PERF-04	Observability screens shall not block main workflows.
NFR-PERF-05	Storefront messaging protected by throttling.

2.6.4 Security

ID	Requirement
NFR-SEC-01	Protected operations require auth and RBAC.
NFR-SEC-02	Credentials not exposed in client-side logs.
NFR-SEC-03	Enforce tenant/store boundary controls.
NFR-SEC-04	Log security events (failures, denials) for audit.
NFR-SEC-05	Webhooks accepted only after signature verification.
NFR-SEC-06	Gmail auto-replies guarded against abuse.

2.7 Requirement Coverage Notes

- Functional and non-functional requirements are implementation-aware but documented at behavior level.
- Documentation language may generalize internal details for readability.
- Future extensions should append new requirement IDs to maintain consistency.

Chapter 3

System Overview

This section describes how JUNO is structured and why the system is decomposed into its current modules. It focuses on architecture, design models, data design, and the domain model, with implementation-aligned detail suitable for technical documentation and evaluation.

3.1 Architectural Design

JUNO is implemented as a role-aware, multi-module platform on top of Next.js App Router. The system combines:

- Role-scoped application surfaces (Vendor, Supplier, Admin).
- Workflow APIs for collaboration and commerce synchronization.
- AI orchestration for storefront/email support.
- Operational observability for governance.

The architecture favors a layered, service-oriented style inside one deployable application boundary, rather than fragmented microservices. This keeps coordination cost low while preserving module boundaries in code and data models.

3.1.1 High-Level Box and Line View

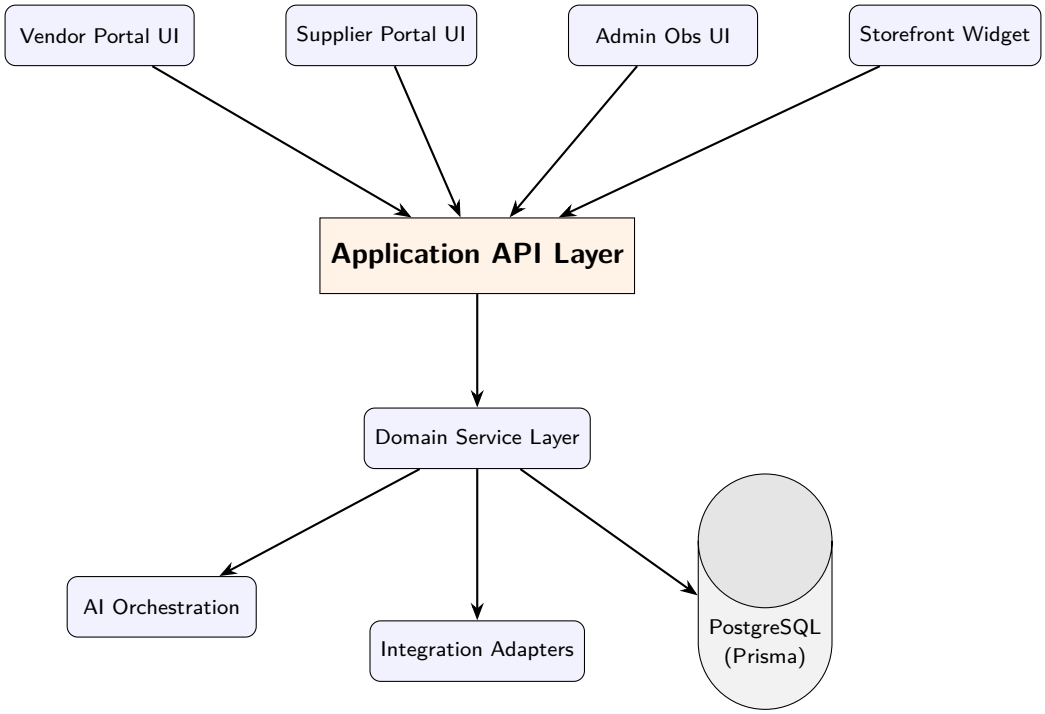


Figure 3.1: High-Level Box and Line View

3.1.2 Layered Architecture Mapping

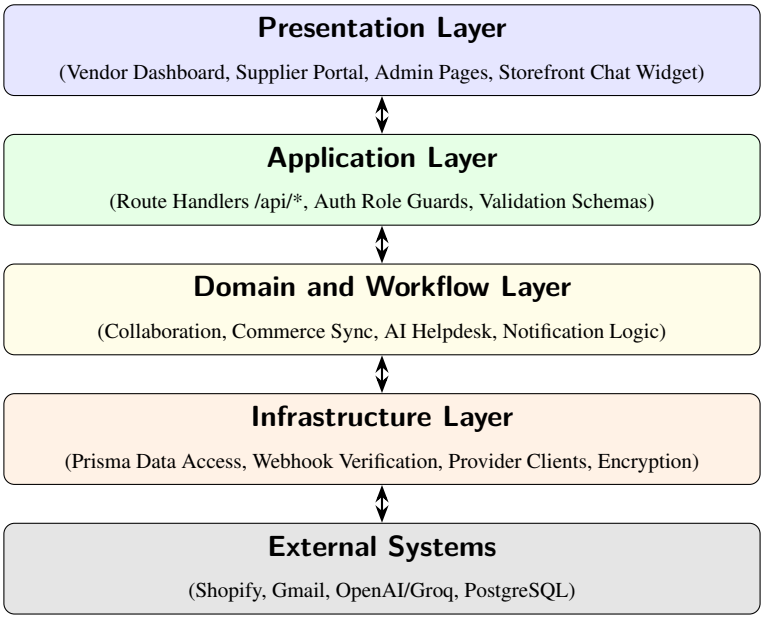


Figure 3.2: Layered Architecture Mapping

3.1.3 Major Module Responsibilities

Table 3.1: Major Module Responsibilities

Module	Primary Responsibilities	Core Artifacts
Identity and Access	Authentication, role checks, portal/session alignment, MFA-aware flows	Auth handlers, role guards, user/session models
Vendor-Supplier Collaboration	Connection lifecycle, product sync requests, supplier offers, event notifications	`Connection`, `ProductSync`, `SupplierProductOffer`
Commerce Synchronization	Shopify webhook verification, cache updates, sync metadata, low-stock signals	`CachedProduct`, `CachedOrder`, `SyncMetadata`
AI Helpdesk and Automation	Storefront AI responses, Gmail ingestion/auto-reply, escalation and ticket routing	Storefront chat flows, email processor, ticket logs
Admin Observability	Cross-module event aggregation and health monitoring	Admin event/sync APIs and monitoring pages

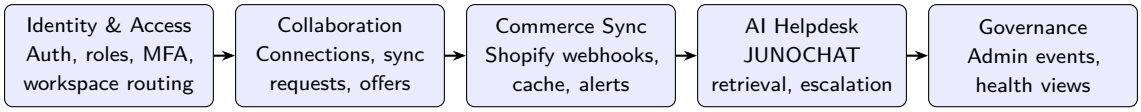


Figure 3.3: Implemented module map (identity, collaboration, commerce sync, AI helpdesk, governance). Arrows indicate typical operational coupling, not a strict pipeline.

3.1.4 Architecture Design Rationale

- **Single deployable with clear module boundaries:** easier release management while preserving separation of concerns.
- **Data-centric workflow design:** collaboration and support are linked through shared store-scoped entities.
- **Role-gated API access:** business-critical operations are protected by actor-specific authorization behavior.
- **Fail-safe automation posture:** AI and webhook pipelines prioritize safety, idempotency, and operator takeover paths.

3.2 Module Operational Design (DFD and State Flows)

3.2.1 Identity, Access, and Workspace Module

This state flow diagram illustrates the lifecycle of a user session, beginning from an unauthenticated state through role-based access control. Upon successful login, the system resolves the user's specific role (Vendor, Supplier, or Admin) to route them to their respective workspace. The active session remains until the user logs out or the session expires, at which point the system reverts to the initial unauthenticated state.

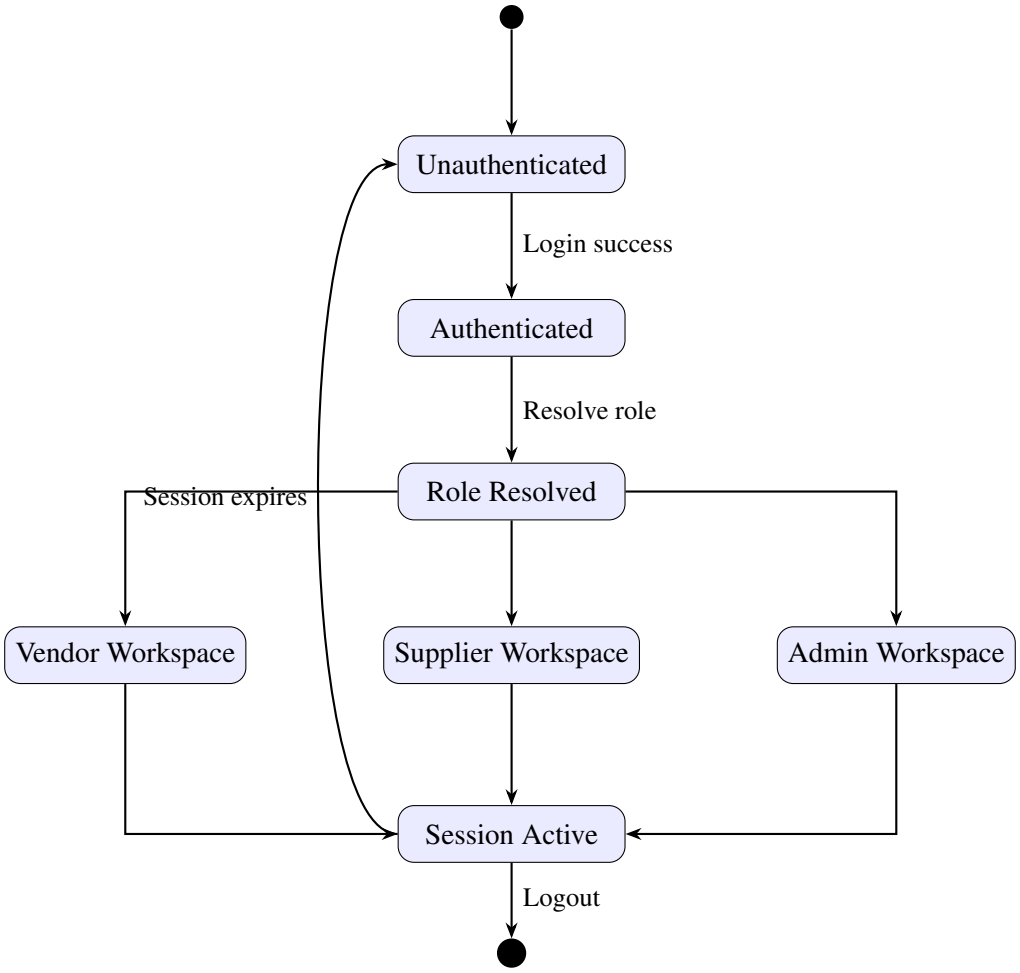


Figure 3.4: Identity and Access State Flow

3.2.2 Commerce Sync and Data Flow (DFD)

This diagram illustrates how external Shopify webhooks are ingested, verified, and parsed to update the internal commerce cache and trigger operational alerts.

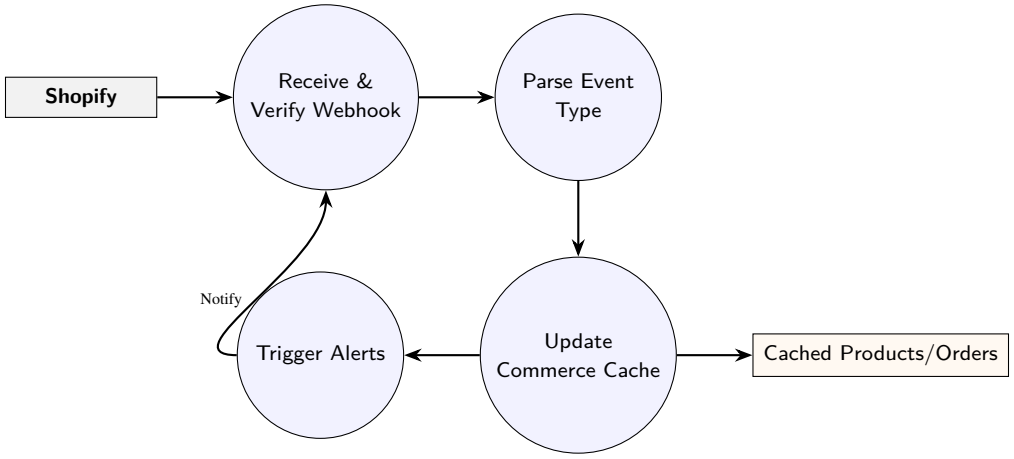


Figure 3.5: Data Flow Diagram: Commerce Sync Module (HMAC-verified Shopify events into the internal cache).

3.3 Process and Interaction Models

3.3.1 Activity Diagram: Vendor Product Sync Lifecycle

The vendor starts by requesting a product sync, which the system immediately kills if there isnt an active connection. Once the connection is verified, a pending request is created and sent over to the supplier for a manual review. The supplier then chooses to either approve or decline the sync, and the system logs that specific state change.

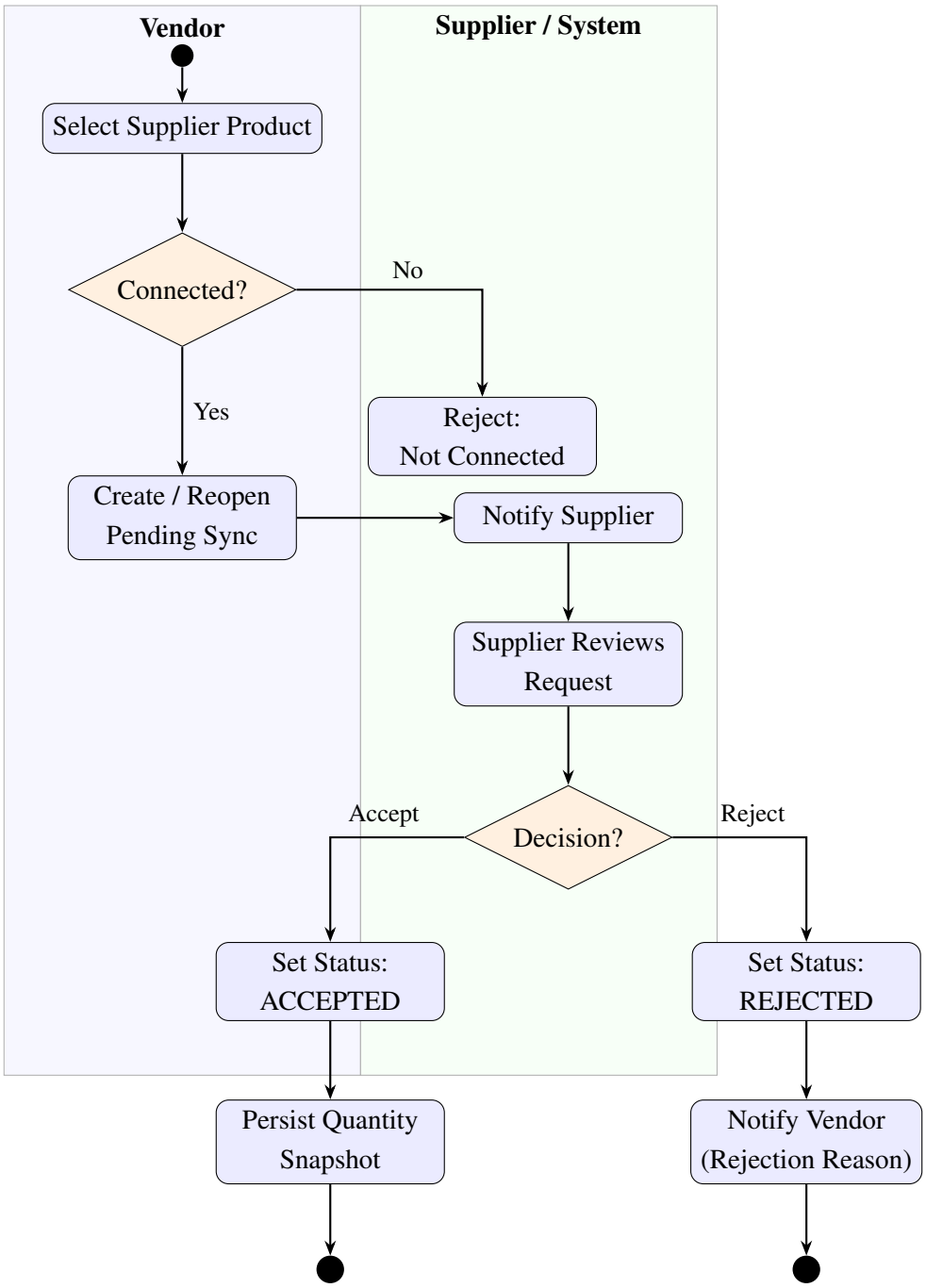


Figure 3.6: Vendor Product Sync Lifecycle Activity Diagram

3.3.2 Sequence Diagram: AI Helpdesk and Escalation

This interaction model describes the *implemented* path from a storefront message to a retrieval-backed draft reply and optional human-readable ticket. `AI Orch` is the orchestration/retrieval service; the Chat API and operator-facing surfaces consume it.

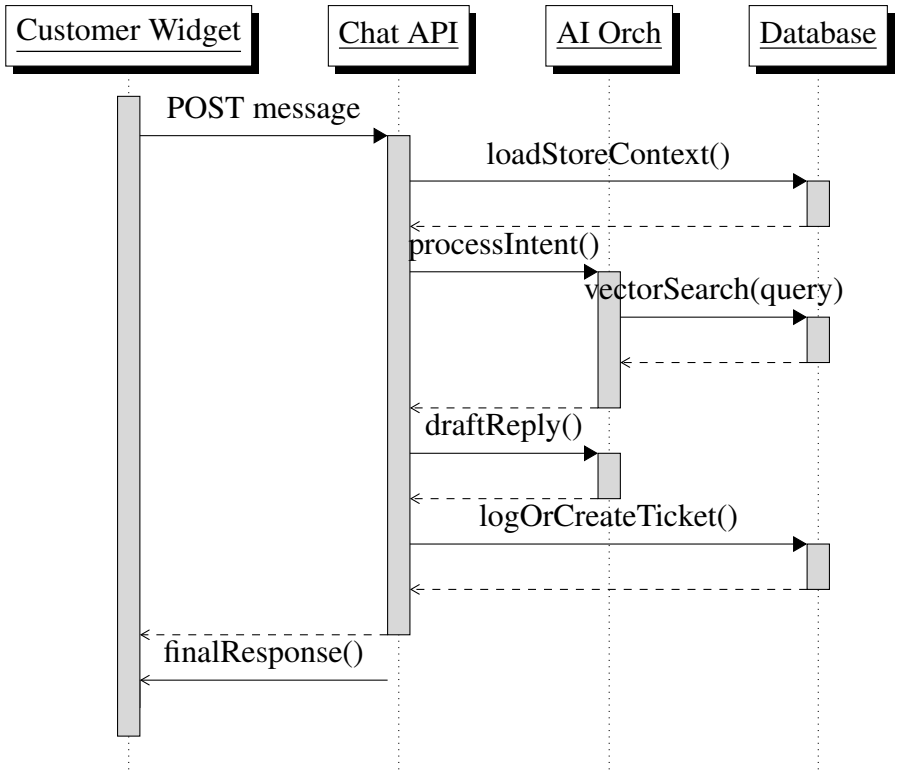


Figure 3.7: AI helpdesk sequence: storefront message, RAG context lookup, draft reply, and optional escalation ticket.

3.4 Data Design

JUNO transforms business interactions into store-scoped, role-aware persistent entities. The data model is implemented in PostgreSQL via Prisma.

3.4.1 Data Storage Strategy

Table 3.2: Data Storage Strategy

Storage Item	**Type**	**Purpose**
PostgreSQL relational tables	Structured	Users, stores, sync records, offers, logs
JSON columns	Semi-structured	Shopify payloads, module policies, metadata
Vector column (`pgvector`)	Embedding	Knowledge chunk retrieval for RAG
External provider state	Integration	Gmail OAuth, Shopify synchronization inputs

3.4.2 Data Integrity and Consistency Design

- **Scoped uniqueness constraints** prevent duplicate collaboration records.
- **Indexing strategy** favors high-frequency filters such as `storeId`.
- **Upsert patterns** are used in sync paths to avoid race-prone duplicate state rows.
- **Idempotent safeguards** reduce duplicate outbound side effects in email automation.

3.5 Domain Model

The JUNO domain model represents operational relationships between merchants, suppliers, and automation.

3.5.1 Conceptual Domain ER Diagram

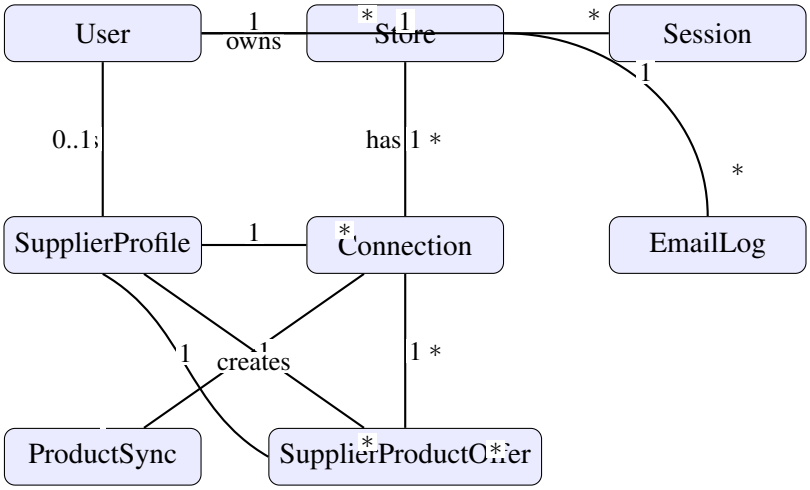


Figure 3.8: Conceptual Domain ER Diagram

Chapter 4

Implementation and Testing

This section explains how JUNO is implemented and validated. It focuses on practical algorithms used in key modules, third-party APIs/SDKs integrated in production workflows, and testing strategy with project-specific evidence.

4.1 Implementation Overview

JUNO is implemented as a full-stack Next.js application with App Router route handlers, Prisma-managed PostgreSQL data access, and modular AI/integration services. The implementation emphasizes:

- Role-based execution paths (Vendor, Supplier, Admin).
- Store-scoped workflow integrity.
- Automation safety and observability.
- Deterministic state transitions for collaboration operations.

Core implementation domains:

- **Collaboration engine:** vendor-supplier connection and product sync lifecycle.
- **Commerce synchronization:** Shopify webhook ingestion and cache/sync metadata updates.
- **AI support automation:** storefront and Gmail pipelines with escalation and safeguards.
- **Operational governance:** admin-level event and sync health visibility.

4.2 Algorithm Design

This subsection documents representative algorithms that drive major runtime behavior.

4.2.1 Algorithm A: Product Sync Request Creation and Reopen Logic

Objective: Create a new sync request only when business preconditions are valid, while safely reusing existing rejected records.

```
Algorithm ProductSyncRequest(vendorUser, supplierId, productPayload):
1. Authorize vendor session and resolve active store.
2. Validate supplier exists and vendor-store has CONNECTED relationship
with supplier.
3. Query existing sync by unique key (storeId, supplierId, shopifyProductId).
4. If existing status is ACCEPTED: reject request ("already synced").
5. If existing status is PENDING: reject request ("already pending").
6. If existing status is REJECTED:
   a. Update same row: status <- PENDING, reset rejected/accepted timestamps.
   b. Refresh product snapshot fields.
   c. Notify supplier.
   d. Return success.
7. Else create new ProductSync row with status PENDING and requester metadata.
8. Emit supplier notification.
9. Return created sync object.
```

4.2.2 Algorithm B: Supplier Decision State Transition

Objective: Enforce valid state transition from `PENDING` to `ACCEPTED/REJECTED` with actor ownership checks.

```
Algorithm DecideSyncRequest(supplierUser, syncId, action):
1. Authorize supplier role and resolve supplier profile.
2. Load ProductSync(syncId) and verify supplier ownership.
3. If status != PENDING: reject ("already processed").
4. If action == ACCEPT:
   a. status <- ACCEPTED
   b. acceptedAt <- now
   c. derive supplier_quantity snapshot from product variants (if available)
5. Else if action == REJECT:
   a. status <- REJECTED
   b. rejectedAt <- now
6. Persist update.
7. Notify vendor about decision.
8. Return updated status response.
```

4.2.3 Algorithm C: Shopify Webhook Verification and Sync Metadata Update

Objective: Process only authentic webhooks and maintain synchronization health state.

```
Algorithm ProcessShopifyWebhook(headers, rawBody):
1. Extract hmac, topic, and domain headers; fail fast if missing.
2. Resolve store by normalized domain variants.
3. Verify HMAC using store-specific API secret.
4. If verification fails: return unauthorized.
5. Switch by topic:
   - orders/create or orders/updated:
     update order cache; upsert SyncMetadata(resource="orders",
     status="success").
   - products/create or products/update:
     update product cache;
     trigger low-stock evaluation;
     attempt auto-link of supplier offers;
     upsert SyncMetadata(resource="products", status="success").
   - products/delete:
     remove cached product;
     upsert SyncMetadata(resource="products", status="success").
6. Log processing outcome and return success.
```

4.2.4 Algorithm Interaction Diagram

The diagram below visualizes the flow of data from external triggerssuch as vendor actions or Shopify eventsthrough the core algorithmic processing layers. It highlights how the application separates commerce synchronization from AI-driven communication while maintaining a shared persistence layer.

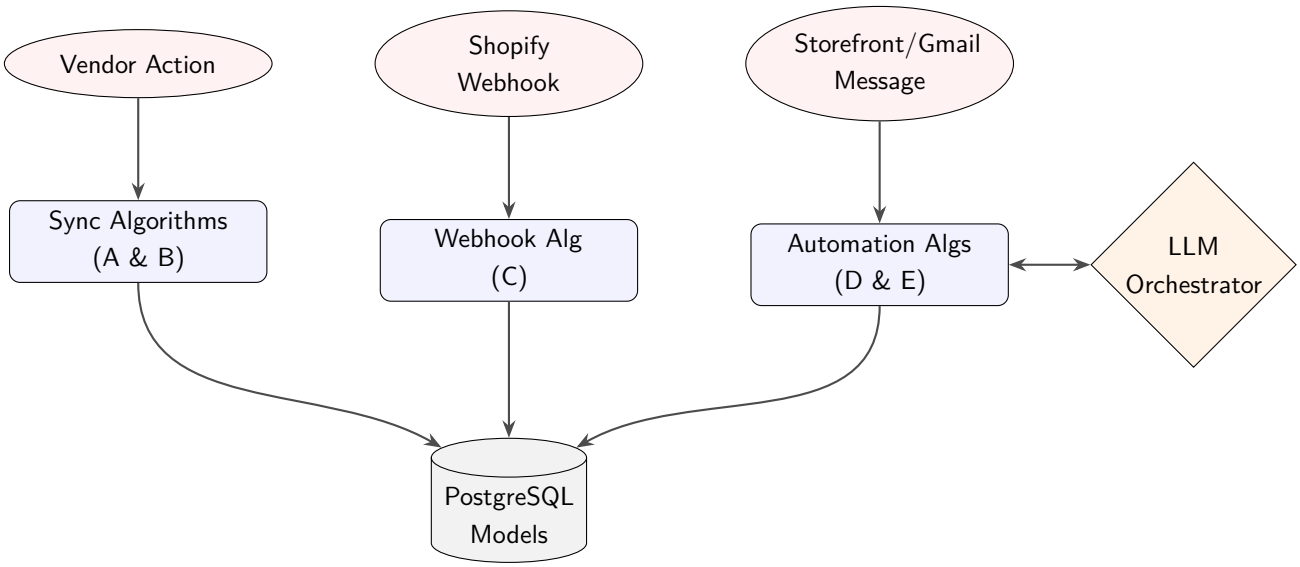


Figure 4.1: Interaction between System Events and Implementation Algorithms

4.3 External APIs and SDKs

JUNO integrates multiple third-party services. Table 4.1 lists major external dependencies used directly in implementation workflows.

Table 4.1: External APIs and SDKs

API/SDK and Version	Description	Purpose in JUNO	Example Usage
Shopify Admin API (2024-01)	Commerce platform API	Store onboarding, webhooks, data sync	`POST /admin/api/webhooks.json`
Gmail API (OAuth2)	Email integration	Vendor email automation	`users/me/messages`
NextAuth/Auth.js	Auth framework	Multi-role session management	Role-aware auth resolution
Prisma ORM	DB access/schema	Type-safe data access, migrations	`prisma.*` model operations
Upstash Rate Limit	Throttling primitives	Public chat anti-abuse protection	`applyRateLimit()`
AWS SDK S3	Object storage	Media upload path for assets	S3 client calls for images
Nodemailer	Email transport	Outbound delivery and logging	Email send service + `EmailLog`
Otplib	TOTP library	MFA verification	`verify({ token, secret })`

4.3.1 Local Model Training and Private Deployment Pipeline (future work)

The **shipped** JUNO helpdesk uses configurable external providers (OpenAI/Groq) with retrieval over store-scoped knowledge. A locally hosted LLM is **not** part of the implemented system. The notes below record a planned privacy-preserving path: instead of routing sensitive vendor–supplier communications to external APIs, a later revision could host a specialised model on premises.

4.3.1.1 Candidate methodology: PEFT via LoRA

If that path is pursued, Low-Rank Adaptation (LoRA) is a candidate technique to limit computational overhead. Weight updates ΔW would be decomposed into two low-rank matrices A and B:

$$\Delta W = B \cdot A$$

where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d,k)$. The forward pass calculation becomes:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r}(B \cdot A)x$$

where α is a constant scaling hyperparameter.

4.4 Testing Details

JUNO testing combines unit tests, integration-style tests, and targeted scenario suites for high-risk automation paths.

4.4.1 Testing Workflow

The testing pipeline is designed to enforce quality at every stage.

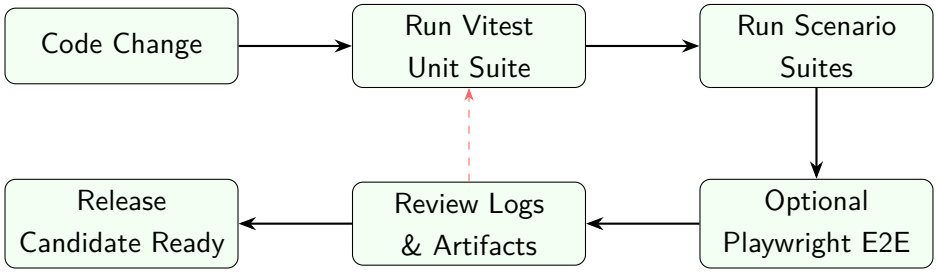


Figure 4.2: JUNO Verification and Testing Workflow

4.4.2 Test Evidence Table

Table 4.2: Test Evidence Table

Test ID	**Objective**	**Method**	**Expected Result**
T-API-AUTH-01	Unauthorized actor cannot post chat payload	Route test (mocked session)	HTTP 403 returned
T-SYNC-STATE-01	Rejected sync can be reopened correctly	Workflow test	Row reused with `PENDING` status
T-WEBHOOK-SEC-01	Invalid Shopify signature is rejected	Security test	Request denied; no mutation
T-AI-ESC-01	Escalation tickets created on signal	AI integration test	`EmailLog` record created
T-GMAIL-SAFE-01	Abusive email does not get auto-reply	Scenario test	Email flagged; no reply sent

4.4.3 QA Test Execution Summary

Table 4.3: QA Test Execution Summary

Metric	Value	Target	Status	Remarks
Total Test Cases Planned	120	N/A	Tracked	Covers core modules
Passed	101	≥ 95	Met	Stable for release candidate
Pass Rate	88.6%	$\geq 85\%$	Met	Calculated on executed tests

4.4.4 Detailed Test Cases

This section provides a granular breakdown of the testing performed on JUNO's core modules. These test cases validate the functional integrity of algorithms, security boundaries, and AI automation logic.

Table 4.4: Test Case: Role-Based Access Control (RBAC) Integrity

Test ID	TC-IAM-01
Feature	Identity and Access Management
Pre-conditions	User is logged in with a 'Vendor' role; Admin dashboard URL is known.
Steps	1. Attempt to navigate directly to `/admin/dashboard`. 2. Attempt to call an Admin-only API endpoint (e.g., `/api/admin/metrics`).
Expected Result	System detects role mismatch, redirects user to Vendor home, and returns HTTP 403 for the API call.
Actual Result	Pass

Table 4.5: Test Case: Product Sync Request Reopen Logic (Algorithm A)

Test ID	TC-COLLAB-01
Feature	Vendor-Supplier Collaboration (Algorithm A)
Pre-conditions	A sync request for Product X already exists with status `REJECTED`.
Steps	1. Vendor submits a new sync request for the same Product X. 2. Observe the database row for the product sync ID.
Expected Result	No new row is created. The existing `REJECTED` row is updated to `PENDING` and timestamps are reset.
Actual Result	Pass

Table 4.6: Test Case: Supplier Decision State Transition (Algorithm B)

Test ID	TC-COLLAB-02
Feature	Supplier Decision Flow (Algorithm B)
Pre-conditions	A sync request exists in `PENDING` state.
Steps	1. Supplier clicks 'Accept' on the dashboard. 2. Check product availability snapshot in the database.
Expected Result	Status changes to `ACCEPTED`, `acceptedAt` is populated, and supplier quantity is snapshotted.
Actual Result	Pass

Table 4.7: Test Case: Shopify Webhook HMAC Verification (Algorithm C)

Test ID	TC-SYNC-01
Feature	Commerce Sync Security (Algorithm C)
Pre-conditions	Shopify Webhook endpoint is active.
Steps	1. Send a POST request to `/api/webhooks/shopify` with a spoofed `X-Shopify-Hmac-Sha256` header.
Expected Result	Algorithm C fails HMAC validation; returns HTTP 401 Unauthorized; no data mutation occurs.
Actual Result	Pass

Table 4.8: Test Case: AI RAG-based Support Response

Test ID	TC-AI-01
Feature	AI Helpdesk (JUNOCHAT)
Pre-conditions	Knowledge base contains shipping policy document.
Steps	1. Customer asks: "What is your return policy for international orders?" 2. Review generated AI response and citations.
Expected Result	System retrieves policy from vector DB and provides a response specific to the store's international shipping rules.
Actual Result	Pass

Table 4.9: Test Case: Automated Ticket Escalation

Test ID	TC-AI-02
Feature	AI Support Automation
Pre-conditions	Support automation is enabled.
Steps	1. Customer sends an angry email about a "broken product" (High-risk intent). 2. Monitor the `EmailLog` and Ticket dashboard.
Expected Result	System classifies message as high-risk, bypasses auto-reply, and creates an escalation ticket for human review.
Actual Result	Pass

Table 4.10: Test Case: Gmail Webhook Auto-Reply Pipeline

Test ID	TC-AI-03
Feature	Gmail Integration
Pre-conditions	Gmail OAuth2 connection is healthy.
Steps	1. Receive an inbound order status query via Gmail. 2. Verify if the system sends an automated draft/reply.
Expected Result	System identifies order ID from message, fetches status from Shopify cache, and sends an automated reply.
Actual Result	Pass

Table 4.11: Test Case: Multi-Factor Authentication (MFA) Verification

Test ID	TC-IAM-02
Feature	Identity Security (otplib integration)
Pre-conditions	MFA is enabled for the user account.
Steps	1. Enter correct credentials. 2. Enter an expired or incorrect 6-digit TOTP code.
Expected Result	Access is denied; error message "Invalid MFA code" is displayed.
Actual Result	Pass

Table 4.12: Test Case: Shopify Store Onboarding and App Proxy

Test ID	TC-IAM-03
Feature	Workspace Setup
Pre-conditions	Vendor has a valid Shopify store URL.
Steps	1. Initiate "Add Store" flow. 2. Input API credentials and verify webhook registration.
Expected Result	Store row is created; Shopify Webhook verification succeeds; Store-scope is active for the vendor.
Actual Result	Pass

Table 4.13: Test Case: Sync Metadata Tracking and Failure Logging

Test ID	TC-SYNC-02
Feature	Operational Governance
Pre-conditions	Shopify Webhook trigger occurs.
Steps	1. Simulate a database connection timeout during webhook processing. 2. Check `SyncMetadata` table.
Expected Result	System catches error, records `status="failed"` in metadata, and logs event for Admin visibility.
Actual Result	Pass

Table 4.14: Test Case: Vendor-Specific Product Offer Creation

Test ID	TC-COLLAB-03
Feature	Supplier Offer Management
Pre-conditions	Supplier is connected to Vendor A.
Steps	1. Supplier creates an offer for Product Y with price $10 for Vendor A. 2. Verify visibility of this offer for Vendor B.
Expected Result	Vendor A sees the $10 offer; Vendor B does not see the offer (Isolation check).
Actual Result	Pass

Table 4.15: Test Case: Anti-Abuse Rate Limiting

Test ID	TC-AI-04
Feature	Public Chat Protection (Upstash)
Pre-conditions	Upstash Rate Limit is configured.
Steps	1. Send 20 storefront chat messages within 10 seconds from the same IP.
Expected Result	After the threshold (e.g., 5 messages), the system returns HTTP 429 (Too Many Requests).
Actual Result	Pass

Table 4.16: Test Case: Admin Event Stream Visibility

Test ID	TC-ADMIN-01
Feature	Admin Observability
Pre-conditions	Multiple system events (logins, syncs, AI replies) have occurred.
Steps	1. Log in as Admin and navigate to Event Stream. 2. Filter by "Sync Failure" topic.
Expected Result	Admin sees a chronological list of all relevant failure events with entity IDs for debugging.
Actual Result	Pass

Chapter 5

Conclusions and Future Work

5.1 Conclusion

JUNO was developed to solve a practical operations gap in Shopify-centered dropshipping workflows: fragmented communication, weak vendor-supplier alignment, and repetitive support overhead. The implemented system demonstrates that these concerns can be addressed effectively through one role-based platform that combines commerce coordination and AI-assisted support.

Across the completed chapters, the project establishes four core outcomes:

- **Operational unification:** Vendor, Supplier, and Admin workflows are modeled in one system instead of disconnected tools.
- **Collaboration formalization:** Product sync and offer workflows are represented as explicit stateful records, improving traceability and accountability.
- **Support automation with safeguards:** Storefront and Gmail automation reduce repetitive workload while preserving human escalation for sensitive cases.
- **Observability and governance:** Admin-facing event and sync health views provide a practical operational monitoring layer.

From an engineering perspective, the implementation aligns design intent with actual behavior:

- Role-aware authorization boundaries are enforced on critical APIs.
- Webhook and integration paths are guarded and auditable.
- Data structures support both transactional integrity and AI context requirements.
- Testing includes deterministic unit coverage and scenario-style automation validation.

The project therefore meets its central objective: delivering a functional, extensible plat-

form that improves dropshipping operations by connecting business collaboration, support intelligence, and operational visibility in a single workflow-oriented architecture.

5.2 Project Limitations

Although the current implementation is production-oriented in many areas, the following limitations remain:

- Channel coverage is not fully unified beyond current storefront and Gmail-centered automation flows.
- AI behavior still depends on provider quality, prompt strategy, and retrieval context freshness.
- Enterprise-grade analytics/reporting depth is limited compared to dedicated BI platforms.
- Some high-throughput ingestion and heavy workflow paths can benefit from more asynchronous processing and queue-based decoupling.

These constraints are expected for the current project scope and do not undermine the core operational value delivered.

5.3 Future Work

Future work is organized into technical, operational, and product expansion tracks.

5.3.1 Multi-Channel Communication Expansion

- Extend support automation beyond current channels to include WhatsApp, Instagram/Messenger, and additional commerce messaging surfaces.
- Maintain a unified conversation identity model across channels to preserve context continuity.
- Add channel-specific policy templates while keeping one escalation governance flow.

5.3.2 Collaboration and Supply Intelligence

- Add proactive supplier recommendation mechanisms based on order trends, fulfillment outcomes, and historical offer quality.
- Support deeper negotiation and version tracking for vendor-specific offers.
- Introduce margin-risk and stock-risk indicators to improve vendor decision support before product commitments.

5.3.3 Reliability and Scalability Enhancements

- Move intensive integration/AI paths to explicit queue-worker architecture for stronger throughput guarantees.
- Expand idempotency and replay protections for all external event ingestion paths.
- Improve resilience testing with fault-injection scenarios for provider outages and delayed external responses.

5.3.4 On-Premises Model Adaptation

- Evaluate a locally hosted LLM with PEFT/LoRA so sensitive vendor–supplier context need not leave the deployment boundary (not implemented in the current system).

5.4 Final Closing Statement

JUNO provides a strong foundation for intelligent commerce operations in dropshipping environments. Its current implementation proves the viability of combining role-based collaboration, synchronized commerce context, and guarded AI automation in one coherent system. With the outlined future work, JUNO can evolve from an operational platform into a broader decision-support and governance ecosystem for modern multi-actor commerce businesses.

Bibliography

[1] Dave Chaffey. *Digital Business and E-commerce Management*. Pearson UK, 2019.

[2] David F. Ferraiolo, Ravi Sandhu, Serban Gavrila, et al. Proposed nist standard for role-based access control. *ACM Transactions on Information and System Security (TISSEC)*, 4(3):224–274, 2001.

[3] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. University of California, Irvine, 2000.

[4] Google. Gmail api overview, 2024. Accessed: 2026-05-08.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[6] Moutaz Khouja and Agni Stergiou. Dropshipping and information sharing in supply chains. *Transportation Research Part E: Logistics and Transportation Review*, 71:45–63, 2014.

[7] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, Inc., 2017.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[9] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.

[10] Microsoft. Playwright: Fast and reliable end-to-end testing, 2024. Accessed: 2026-05-08.

[11] Serguei Netessine and Nils Rudi. Supply chain structures on the internet: Marketing-operations interactions. *Management Science*, 52(1):15–31, 2006.

[12] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Inc., 2021.

[13] Inc. Prisma Data. Prisma orm: Type-safe database client, 2024. Accessed: 2026-05-08.

[14] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC Editor, 2018.

[15] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *Computer*, 29(2):38–47, 1996.

[16] Shopify. Shopify admin api reference, 2024. Version: 2024-01.

[17] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[18] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[19] Vercel. Next.js documentation, 2024. Accessed: 2026-05-08.

[20] Vitest. Vitest: Next generation testing framework, 2024. Accessed: 2026-05-08.

Appendix A

Supplementary Material

This appendix reprints team ownership, a short glossary of terms used in the report, a compact index of documented test identifiers, and exported architecture views already present in the project source. It does not introduce new product features.

A.1 Work Division Reprint

Table A.1 repeats Table 1.3 for standalone reading of the back matter.

Table A.1: Work Division (reprint)

Team Member	**Responsibilities**
Syed Saad Mohsin	Role-gated vendor/supplier/admin interfaces and UX; JUNOCHAT integration into operator workflows; UI/functional, security, and AI end-to-end testing.
Muhammad Usman	AI orchestration, retrieval/provider behaviour, and notifications; Prisma schema, relations, migrations, and data consistency.
Talat Faheem	APIs, business workflows, webhook handlers; deployment, environment, and CI/CD operations.

A.2 Glossary

RBAC Role-based access control. JUNO gates vendor, supplier, and admin operations by actor role and store scope.

RAG Retrieval-augmented generation. JUNOCHAT retrieves store-scoped knowledge chunks (including vector search) before drafting a support reply.

HMAC
Hash-based message authentication code. Used to verify inbound Shopify web-

hooks before cache mutation.

MFA Multi-factor authentication. TOTP-based second factor on identity flows.

JUNOCHAT
The implemented AI helpdesk module (storefront chat, email automation, escalation).

A.3 Selected Test Identifier Index

Identifiers below appear in Chapter 4. They index documented cases; they are not a claim of additional unpublished features.

Table A.2: Selected test identifiers

ID	Focus
TC-IAM-01, TC-IAM-02, TC-IAM-03	Identity, RBAC, and MFA paths
TC-COLLAB-01, TC-COLLAB-02, TC-COLLAB-03	Vendor–supplier collaboration workflows
TC-SYNC-01, TC-SYNC-02	Commerce sync and webhook ingestion
TC-AI-01	RAG-based storefront support response
TC-AI-02	AI escalation to a human-readable ticket
TC-AI-03	Gmail auto-reply / safety path
TC-AI-04	Related helpdesk automation UAT
TC-ADMIN-01	Admin observability / event visibility
T-WEBHOOK-SEC-01	Invalid Shopify signature rejected
T-AI-ESC-01, T-GMAIL-SAFE-01	Escalation record and abusive-email safeguard

A.4 Exported Architecture Views

Figures A.1 and A.2 are source exports of the same commerce/support data-flow views used while documenting the implemented modules (TikZ equivalents appear in Chapters 1 and 3).

Data Flow Diagram

Data flow diagram symbols, symbol names, and examples

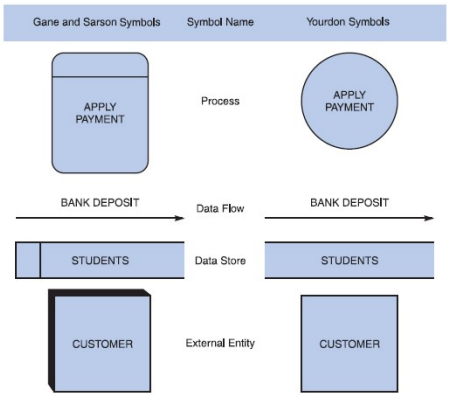


Figure B-5 Data flow diagram symbols, symbol names, and examples of the Gane and Sarson and Yourdon symbol sets.

Guidelines for Drawing DFDs

Step 1: Draw a Context Diagram: The first step in constructing a set of DFDs is to draw a context diagram. A **context diagram** is a top-level view of an information system that shows the system's boundaries and scope. Data stores are not shown in the context diagram because they are contained within the system and remain hidden until more detailed diagrams are created.

Example

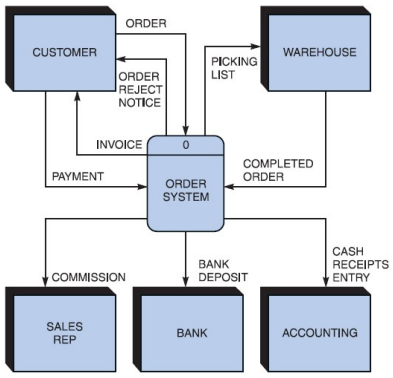


Figure B-6 Context diagram DFD for an order system.

Figure A.1: Exported data-flow view of JUNO modules (source export).

Step 2: Draw a Diagram 0 DFD: To show the detail inside the black box, you create DFD diagram 0. **Diagram 0** zooms in on the system and shows major internal processes, data flows, and data stores. Diagram 0 also repeats the entities and data flows that appear in the context diagram. When you expand the context diagram into DFD diagram 0, you must retain all the connections that flow into and out of process 0.
Example

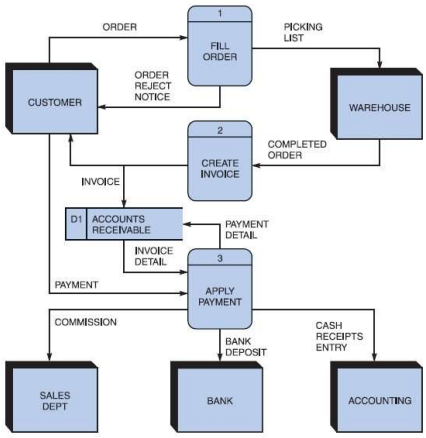


Figure B-7 Diagram 0 DFD for the order system.

Step 3: Draw the Lower-Level Diagrams:
To create lower-level diagrams, you must use leveling and balancing techniques. **Leveling** is the process of drawing a series of increasingly detailed diagrams, until all functional primitives are identified.
Leveling Example

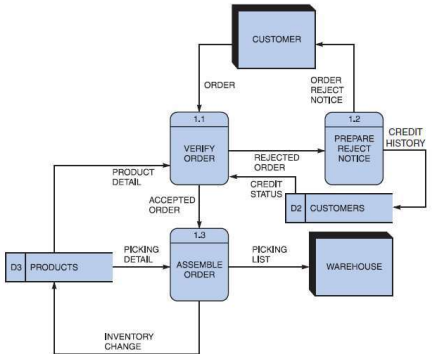


Figure B-8 Diagram 1 DFD shows details of the FILL ORDER process in the order system.

Figure A.2: Exported data-flow view (alternate layout; source export).